Campaign Events — Engineering Walkthrough

This document explains **what was built, why, how it was verified, and how to talk about it in an interview**. It is written so it can be read top-to-bottom by someone who has never seen the assignment.

---

1. The problem in plain language

Relay sends marketing messages on behalf of clients but does not deliver them; third-party providers do. As things happen to a message, the provider calls Relay's webhook with an **event**:

Event	Meaning
`sent`	the provider accepted the message for delivery
`delivered`	the message reached the recipient
`opened`	the recipient opened it
`clicked`	the recipient clicked a link

The dashboard needs per-campaign answers: how many sent, delivered, opened, clicked.

Two real-world facts make this harder than counting:

1. **Providers retry.** The same event can arrive many times. Counting requests instead of events double-counts.
2. **Events are late and out of order.** An `opened` may arrive before the `delivered` for the same message, hours after it happened.

The job is therefore: **accept messy webhook traffic and maintain trustworthy, per-campaign event counts.**

Mental model:

```
provider -> POST /events -> validate -> de-duplicate by event_id -> store
                                                                     |
dashboard -> GET /campaigns/{id}/stats <----------------------+
```

---

2. Environment and setup

The assignment requires Go 1.22+. In the environment used to build this, Go was not pre-installed, so a local toolchain was downloaded to `/tmp/opencode/go` (no root needed):

```
cd /tmp/opencode
curl -sSL -o go.tar.gz https://go.dev/dl/go1.22.12.linux-amd64.tar.gz
tar -xzf go.tar.gz
export PATH=/tmp/opencode/go/bin:$PATH
go version
```

On a normal machine, install Go normally and check `go version`. No other dependency is needed: the service uses only the standard library.

---

3. Repository layout

```
campaign-events/
├── main.go                      wiring and server startup
├── go.mod                       module campaign-events, go 1.22
├── internal/
│   ├── models/event.go          Event, Stats, types, payload equality
│   ├── repository/
│   │   ├── repository.go        storage interface
│   │   └── memory.go            in-memory concurrency-safe store
│   ├── services/ingest.go       validation, batch handling, per-item results
│   └── handlers/handlers.go     HTTP routes and JSON responses
├── internal/**/*_test.go        unit and integration tests
├── starter/                     supplied skeleton + seed fixture
├── debugging/                   Part 3 program, four fixes applied
├── NOTES.md                     Parts 1, 4, 5 + completion notes
├── BUGS.md                      Part 3 findings
├── AI_USAGE.md                  AI disclosure
├── README.md                    run instructions and API reference
└── DOCUMENTATION.md             this file
```

The layering matters: HTTP knows nothing about storage, storage knows nothing about

HTTP, and validation lives in the service. That is what makes the in-memory store replaceable by SQLite without touching handlers.

---

4. Step-by-step build log

Each step lists **what changed**, **why**, **how to verify**, and a one-line **interview explanation**.

Step 1 — Prepare the submission

What: inspected the supplied files (`ASSIGNMENT.md`, `starter/`, `debugging/`), created `campaign-events/`, and copied `starter/`, `debugging/` and the brief into it.

Why: the submission must be self-contained and must include the fixed `debugging/` folder. Copying instead of moving keeps the original files as a known-good reference.

Verify: `find campaign-events -maxdepth 2 -type f` shows the expected files.

Interview line: "*I made the submission folder self-contained so the service and the debugging fixes ship together, and kept pristine originals for comparison.*"

Step 2 — Model and validation

What: `internal/models/event.go` defines `Event`, `Stats`, the closed set of valid types, and `SamePayload` for conflict detection.

Why: validation needs a single definition of a valid event. `SamePayload` separates "the same event retried" from "the same id reused with different data".

Verify: `go test ./internal/services` covers missing fields, bad types, bad timestamps, and conflicts.

Interview line: "*`event_id` is identity; the rest is payload. If identity repeats with the same payload it is a retry; if the payload differs it is a provider bug and I keep the first version.*"

Step 3 — Storage

What: `internal/repository/repository.go` defines a `Repository` interface; `memory.go` implements it with two maps (`byID`, `byCampaign`) and a `sync.RWMutex`. `Insert` performs an atomic check-then-store on `event_id`.

Why: idempotency must be enforced at the storage boundary, because that is where two concurrent requests race. Keeping `byID` separate makes the uniqueness rule explicit. Stats are computed from stored events rather than maintained as separate counters, so they cannot drift.

Verify: the concurrency test fires ten goroutines ingesting overlapping batches and asserts the final count equals the number of distinct events, under `-race`.

Interview line: "*I made the duplicate check and the insert one critical section, so two simultaneous retries cannot both succeed. I also derive stats from the event log so there is one source of truth.*"

Step 4 — Ingestion service

What: `internal/services/ingest.go` decodes the batch as `[]json.RawMessage`, validates each element, inserts it, and returns a structured per-item result.

Why: decoding into `[]Event` means one bad element fails the whole array. Per-item decoding isolates failures and gives providers actionable feedback. Invalid elements are reported, not silently dropped.

Verify: `TestIngestInvalidEventsDoNotBreakTheBatch` mixes one valid event with eight invalid ones and asserts `accepted=1`, `invalid=8`.

Interview line: "*A batch is a transport detail; validation is per event. One bad event must not erase 199 good ones.*"

Step 5 — HTTP API

What: `internal/handlers/handlers.go` registers three Go 1.22 routes: `POST /events`, `GET /campaigns/{campaignID}/stats`, and `GET /campaigns/{campaignID}/events`. `main.go` wires repository → service → handlers and starts the server.

Why: the handlers stay thin: parse, delegate, serialise. A `10 MiB` body cap prevents unbounded reads. Only a malformed top-level body returns `400`; per-item problems return `200` with a report so providers do not endlessly retry permanent bad data.

Verify: handler tests exercise success, mixed batches, pagination, and all error statuses.

Interview line: "*The API separates transport errors from data errors: malformed HTTP is 400, but a valid batch with bad elements is 200 with a per-item report.*"

Step 6 — Tests

What: tests in `internal/services` and `internal/handlers`.

Why: they cover the behaviour that would be most embarrassing to break: duplicates, conflicts, mixed batches, unique opens, out-of-order events, concurrent ingestion, stats and pagination, and the supplied seed.

Verify:

```
go vet ./...
go test ./...
go test -race ./...
```

All pass. The concurrency test is meaningful only with `-race` (and with many repetitions), which is why both are part of the workflow.

Interview line: "*I test invariants, not coverage. The invariant is: the number of stored events equals the number of distinct valid event ids, no matter how requests interleave.*"

Step 7 — Debugging exercise

What: fixed four bugs in `debugging/main.go` with minimal diffs, documented in `BUGS.md`.

Why / evidence:

#	Bug	Evidence from the original program
1	de-dupe set reset per 200-event batch	`cmp_A sent=2024` vs expected `1987`
2	`unique_opens` keyed only by contact	`cmp_A unique_opens=549` vs expected `942`
3	`.Local()` instead of `.UTC()` for daily buckets	extra `2026-08-08` buckets, days shuffled
4	unsynchronised `++` on shared counters	`go run -race` reports `DATA RACE` in `apply`

Verify:

```
cd debugging
go run . events.jsonl > actual.txt
diff actual.txt expected_output.txt      # empty
go run -race . events.jsonl > /dev/null  # no race report
```

Interview line: "*The scoping bugs changed the numbers; the timezone bug only appears outside UTC; the race bug may not change the numbers at all on a given run, which is why the race detector matters.*"

Step 8 — Written analysis and docs

What: `NOTES.md` (interpretation, assumptions, ambiguities, PM questions, priorities, scale memo, angry-marketer analysis, completion notes), `BUGS.md`, `AI_USAGE.md`, `README.md`, and this document.

Why: the assignment explicitly weights reasoning and honesty about what was and was not done.

---

5. API reference (summary)

`POST /events`

Body: JSON array of events. Response `200`:

```
{
  "received": 4,
  "accepted": 2,
  "duplicates": 1,
  "invalid": 1,
  "results": [
    {"index": 0, "event_id": "evt_1", "status": "accepted"},
    {"index": 1, "event_id": "evt_2", "status": "accepted"},
    {"index": 2, "event_id": "evt_2", "status": "duplicate"},
    {"index": 3, "status": "invalid", "error": "event_id is required"}
  ]
}
```

- `accepted` — first time this `event_id` was stored.
- `duplicate` — already stored; `conflict: true` if the payload differs.
- `invalid` — failed validation; `error` says why.
- Request body not a JSON array → `400`.

`GET /campaigns/{campaign_id}/stats`

```
{
  "campaign_id": "cmp_summer_sale",
```

```
  "sent": 1,
  "delivered": 1,
  "opened": 1,
  "clicked": 0,
  "unique_opens": 1
}
```

`404` if no events are stored for that campaign.

`GET /campaigns/{campaign_id}/events?limit=20&offset=0`

```
{
  "campaign_id": "cmp_summer_sale",
  "total": 3,
  "limit": 2,
  "offset": 0,
  "events": [ ... newest event time first ... ]
}
```

`limit` 1–100 (default 20); `offset >= 0`.

---

6. Design decisions and trade-offs

Question	Decision	Why
What makes two events the same?	Same `event_id`	It is the provider's identity for one event
Duplicate response	`200` + `duplicate`, conflicts flagged	Errors would trigger pointless retries
Invalid element in a batch	Report it, keep the good ones	Isolation; provider gets actionable output
Malformed whole body	`400`	It is not a batch at all
Meaning of "opened"	Event count `opened` + distinct-contact `unique_opens`	Dashboards use both
Storage	In-memory maps behind an	Fast, simple, testable; durable

Question	Decision	Why
	interface	store is a drop-in
Concurrency	One `RWMutex` around check+insert	Correctness first; throughput at this scale is fine
Late/out-of-order	Store event time; never gate on arrival	Counts are set-based and order-independent
Duplicate with different payload	First wins, report conflict	Deterministic and surfaces provider bugs

Known limitation: data is lost on restart. The repository interface and the scale memo describe the durable/queue-based evolution.

---

7. How to run and demo

```
cd campaign-events
go run .                                    # starts on :8080

# in another terminal
curl -s -X POST localhost:8080/events \
  -H 'Content-Type: application/json' \
  -d
'[{"event_id":"evt_1","campaign_id":"cmp_demo","contact_id":"ct_1","type":"sent","timest
amp":"2026-08-10T06:00:00Z"},

{"event_id":"evt_1","campaign_id":"cmp_demo","contact_id":"ct_1","type":"sent","timestam
p":"2026-08-10T06:00:00Z"},

{"event_id":"evt_2","campaign_id":"cmp_demo","contact_id":"ct_1","type":"opened","timest
amp":"2026-08-10T06:05:00Z"}]'

curl -s localhost:8080/campaigns/cmp_demo/stats
curl -s 'localhost:8080/campaigns/cmp_demo/events?limit=1&offset=0'
```

Expected: `accepted=2, duplicates=1`; stats show `sent=1, opened=1, unique_opens=1`.

Load the realistic dirty fixture:

```
curl -s -X POST localhost:8080/events \
```

```
  -H 'Content-Type: application/json' \
  --data-binary @starter/seed/events.json
```

Run the checks:

```
go vet ./...
go test ./...
go test -race ./...
cd debugging && go run . events.jsonl | diff - expected_output.txt
```

---

8. Interview explanations

30-second version

"*I built a Go webhook service that ingests provider events in batches, validates each event independently, and de-duplicates by `event_id` at the storage layer so retries never double-count. Stats are derived from the stored event log, so late and out-of-order events are handled naturally. The storage is behind an interface, so it can move to SQLite or Postgres without changing the API. I also fixed four bugs in a supplied program, including a data race and a UTC bug, and wrote up scaling and anomaly analysis.*"

2-minute version

"*The core difficulty is idempotency plus disorder. Providers retry and send events late, so I cannot count requests or assume arrival order. I treat `event_id` as the provider's idempotency key. The repository performs check-and-insert under one lock, which makes concurrent retries safe. A duplicate is returned as `duplicate` rather than an error, because an error would cause more retries; if the payload differs, I keep the first and flag the conflict so the provider problem is visible. Validation is per item using `json.RawMessage`, so one malformed event does not destroy the batch. Stats are computed from the distinct stored events, which keeps `opened` and `unique_opens` consistent and order-independent. Tests cover duplicates, conflicts, mixed batches, concurrency under `-race`, pagination, and the dirty seed fixture. For scale, memory and O(n) stats scans break first; I would move to Postgres with a unique `event_id`, precomputed counters, and a queue, and I explained the duplicate and counter-consistency problems that introduces.*"

Likely questions and answers

- **Why not return 409 on duplicates?** A duplicate is a normal, expected success from

the provider's perspective. Returning an error teaches the provider to retry forever. I report it in the body instead.

- **Why not update an existing event if the payload differs?** First-wins is deterministic and avoids order-dependent history rewrites. Conflicts are surfaced for investigation.
- **Why compute stats instead of incrementing counters?** One source of truth and no drift. At high volume I would precompute counters transactionally and reconcile against the log.
- **What happens under two simultaneous identical requests?** The critical section in `Insert` means exactly one stores; the other observes the id and returns duplicate.
- **What about late events?** They are stored whenever they arrive; totals are set-based, so lateness changes freshness, not correctness.

---

9. Verification checklist

- [x] `go vet ./...` clean
- [x] `go test ./...` passes
- [x] `go test -race ./...` passes
- [x] Supplied seed loads without a batch-wide failure
- [x] Mixed batch: valid stored, invalid reported
- [x] Duplicate retries counted once
- [x] Conflicting duplicate flagged, first kept
- [x] Per-campaign unique opens correct
- [x] Out-of-order events produce correct totals
- [x] Pagination newest-first with stable ordering
- [x] Debugging output matches `expected_output.txt` exactly
- [x] Debugging output deterministic across runs
- [x] `go run -race` on debugging produces no race report
- [x] README has exact run/test/demo commands